

BASE DE DATOS II · CBS08

Sistema de Inventario de Equipos para Empresa

Universidad Nacional de Ingeniería — Escuela de Ingeniería de Ciberseguridad

**Juan Carlos Matias Gonzales
Avenidaño**

Áreas y Empleados

Alexandro Achalma Galindo

Equipos (CRUD y filtros)

**Carlos Henry Santana
Palomino**

Mantenimiento y Reportes

Antony Jamel Lipa Benito

Arquitectura y Seguridad

Cuatro módulos, un mismo sistema



01 Áreas y Empleados

Modelado referenciado y CRUD base



02 Equipos

CRUD, filtros dinámicos y datos embebidos



03 Mantenimiento y Reportes

Historial embebido y agregaciones



04 Arquitectura y Seguridad

Conexión, validación, roles y despliegue

¿Qué resuelve el sistema y con qué está hecho?

Una empresa necesita controlar sus equipos de cómputo: quién los tiene asignado, en qué área, su estado y su historial de mantenimiento.



ASP.NET Core MVC

Controladores, vistas Razor



MongoDB.Driver (C#)

Acceso fuertemente tipado a la base



Patrón MVC

Modelo – Vista – Controlador

FLUJO DE LA APLICACIÓN

Navegador (Vistas Razor)

El usuario hace clic: crear, editar, filtrar, listar



Controllers (C#)

Area · Empleado · Equipo · Reporte — reciben la petición



MongoDB.Driver

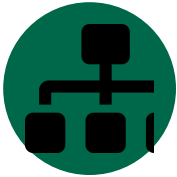
Builders<T>.Filter / Update construyen la operación



MongoDB

Colecciones: areas, empleados, equipos

Módulo de Áreas y Empleados



CRUD completo sobre dos colecciones, con el patrón de modelado referenciado.



Áreas

Colección independiente: nombre y responsable.
CRUD simple, sin relaciones.



Empleados

Cada empleado guarda solo el AreaId — no duplica los datos del área.



Resolución manual

MongoDB no hace JOIN: el controller cruza ambas listas en memoria.

`area_id`

→ único campo que conecta Empleado con Área. No se copian nombre ni responsable dentro del empleado.

Archivos: `Models/Area.cs` · `Models/Empleado.cs` · `Controllers/AreaController.cs` · `Controllers/EmpleadoController.cs`

Resolviendo la referencia en código

EmpleadoController.cs — método Index()

```
var empleados = _empleados.Find(_ => true).ToList();
var areas     = _areas.Find(_ => true).ToList();

foreach (var emp in empleados)
{
    var area = areas.FirstOrDefault(a => a.Id == emp.AreaId);
    emp.AreaNombre = area != null ? area.Nombre : "";
}
```

Por qué así y no con \$lookup

MongoDB no hace JOIN automático. El operador \$lookup no estaba en el temario del curso, así que la relación se resuelve manualmente: se traen ambas colecciones y se cruzan en memoria con C#.

MODELADO REFERENCIADO

Empleado

```
{ "area_id": "665a..." }
```

↓ *referencia por Id*

Área (colección propia)

```
{ "_id": "665a...", "nombre":  
  "Sistemas" }
```

Se comparte entre muchos empleados y equipos sin duplicar datos.

Módulo de Equipos



CRUD de equipos con filtros dinámicos y un campo embebido de especificaciones técnicas.



Filtros dinámicos

Index(areald, estado) arma el filtro solo con lo que el usuario elige, con Builders.Filter.And.



Especificaciones embebidas

Dictionary<string,string> dentro del propio documento: RAM, procesador, etc.



Referencias a Área y Empleado

Areald y AsignadoA conectan el equipo con su área y su responsable.

Archivos: Models/Equipo.cs · Controllers/EquipoController.cs (Index, Create, Edit, Delete) · Views/Equipo/*

Filtros dinámicos con Builders<T>

EquipoController.cs — método Index(areaId, estado)

```
var filtro = Builders<Equipo>.Filter.Empty;

if (!string.IsNullOrEmpty(areaId))
    filtro &= Builders<Equipo>.Filter.Eq(e => e.AreaId, areaId);

if (!string.IsNullOrEmpty(estado))
    filtro &= Builders<Equipo>.Filter.Eq(e => e.Estado, estado);

var equipos = _equipos.Find(filtro).ToList();
```

Idea clave

El filtro empieza vacío y se le agregan condiciones solo si el usuario las envió. Así una misma vista sirve para "ver todo", "ver por área", "ver por estado" o ambos combinados.

ESPECIFICACIONES EMBEBIDAS

Equipo

```
{
  "codigo_inventario": "PC-001",
  "tipo": "laptop",
  "especificaciones": {
    "ram": "16GB",
    "procesador": "i7",
    "almacenamiento": "512GB SSD"
  }
}
```

Historial de mantenimiento embebido



Cada equipo guarda su propio arreglo de mantenimientos — sin colección aparte, sin id propio por registro.

EquipoController.cs — AgregarMantenimiento()

```
var registro = new MantenimientoRegistro {
    Fecha = DateTime.UtcNow,
    TipoMantenimiento = tipoMantenimiento,
    Tecnico = tecnico,
    Costo = costo
};

var filtro = Builders<Equipo>.Filter.Eq(e => e.Id, id);
var update = Builders<Equipo>.Update
    .Push(e => e.HistorialMantenimiento, registro);

_equipos.UpdateOne(filtro, update);
```

\$push

agrega un elemento al final del arreglo sin reescribir todo el documento.

```
historial_mantenimiento: [
  { "fecha": "2025-02-10",
    "tipo": "Limpieza",
    "costo": 30 },
  { "fecha": "2025-08-15",
    "tipo": "Cambio batería",
    "costo": 120 }
]
```

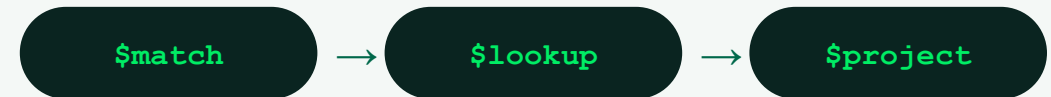
Sin_id propio: pertenece únicamente a este equipo.

Reportes con el framework de agregación

Tres pipelines distintos, cada uno construido con etapas del framework de agregación de MongoDB.

Áreas con su personal y equipos

Trae cada área junto con sus empleados y equipos relacionados.



Equipos agrupados por estado

Cuenta cuántos equipos hay activos, en mantenimiento o de baja.



Costo total de mantenimiento por equipo

Descompone el arreglo embebido y suma los costos de cada equipo.



Arquitectura, conexión y validación



Conexión Singleton

DBConnection se registra una sola vez en Program.cs y se inyecta por constructor en cada controller.



Validación de esquema

\$jsonSchema exige campos obligatorios y tipos correctos antes de guardar.



Índices

Índice único en codigo_inventario evita equipos duplicados; otros índices aceleran las búsquedas.

SEGURIDAD: USUARIOS Y ROLES

`admin_inventario`

root

administra toda la base

`app_inventario`

readWrite

usado por la propia aplicación

`consulta_inventario`

read

solo lectura, para auditoría

Puesta en marcha y demo en vivo

01

Ejecutar scripts

01_setup_database.js crea colecciones con validación; 02_seed_data.js carga datos de prueba.

02

Configurar seguridad

03_security_setup.js crea los 3 usuarios con sus roles y activa --auth.

03

Levantar la aplicación

dotnet run inicia el servidor MVC; DBConnection abre la conexión a MongoDB.

04

Demostrar en vivo

Cambiar el connection string al usuario de solo lectura y mostrar el rechazo al intentar crear un equipo.

Gracias

